

TPU_V4 논문 리뷰

26/01/07 세미나

목차

- 1. Abstract
- 2. Introduction
- 3. TPU v1
- 4. Optical Circuit Switching
- 5. SparseCore
- 6. TPU 관련 지표
- 7. Summary

Abstract

Abstract

- TPU v4: 구글의 다섯 번째 DSA이자 이러한 ML 모델을 위한 세 번째 슈퍼컴퓨터
- 광학 회로 스위치(OCS)의 도입
 1. 동적 재구성
 2. Twisted 3D Torus
 3. 비용 및 전력 효율
- 임베딩 가속기: 스파스코어(SparseCore)
- 거대한 시스템으로서의 성능 극대화
- 경쟁 모델과의 비교

Nvidia A100 대비: 약 1.2~1.7배 빠르고, 1.3배~1.9배 전력 소모 적음

Graphcore IPU Bow 대비: 약 4.3배~4.5배 더 빠른 성능
- 친환경 및 에너지 효율

Introduction

Introduction

- TPU의 세가지 주요 기능
 1. 광학 회로 스위치(OCS) 도입
 2. DLRM의 임베딩에 대한 하드웨어 지원(SparseCore) 공개
 3. 위 두 기능을 결합한 all-to-all 통신 패턴 추가
- 최근 LLM의 규모는 기존 ML 슈퍼컴퓨터 규모를 256개(TPU v2) 노드에서 4096개(TPU v4) 노드로 확장시킨다

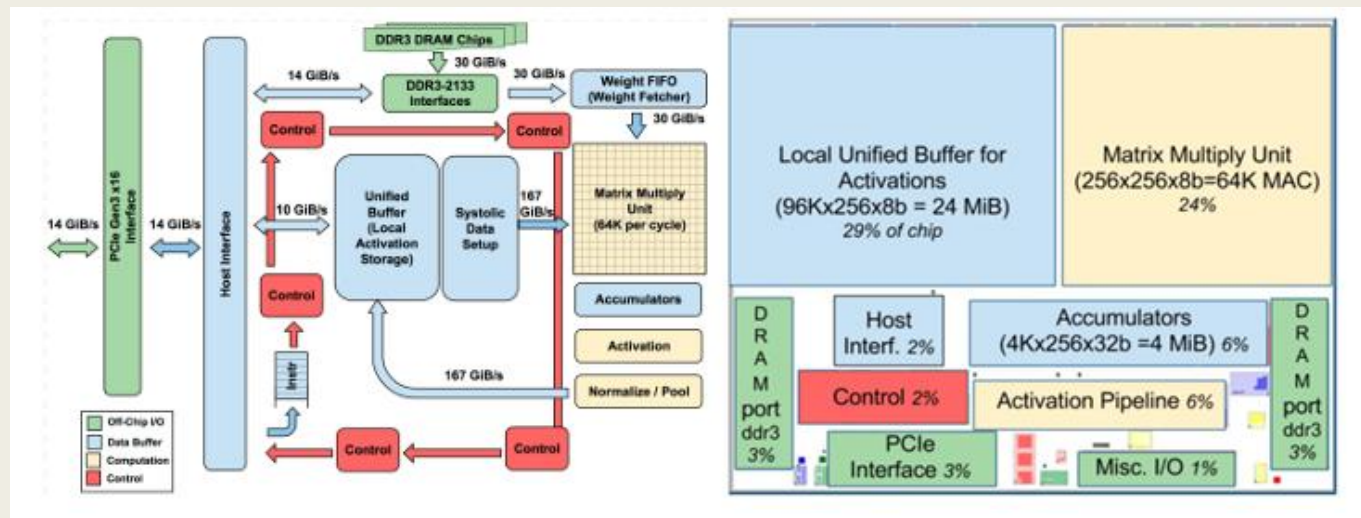
TPU v1

TPU 등장 배경

- DNN 기술의 발전으로 다양한 서비스들이 발전했는데 이로 인해 엄청난 계산 능력이 필요해짐 -> 기존 CPU나 GPU로는 비용과 전력 효율 면에서 한계 있음
-> '신경망 전용' 하드웨어 필요해짐
- TPU는 구글이 신경망 추론 연산이라는 특정 작업을 가속화하기 위해 설계한 맞춤형 칩인 ASIC이다.

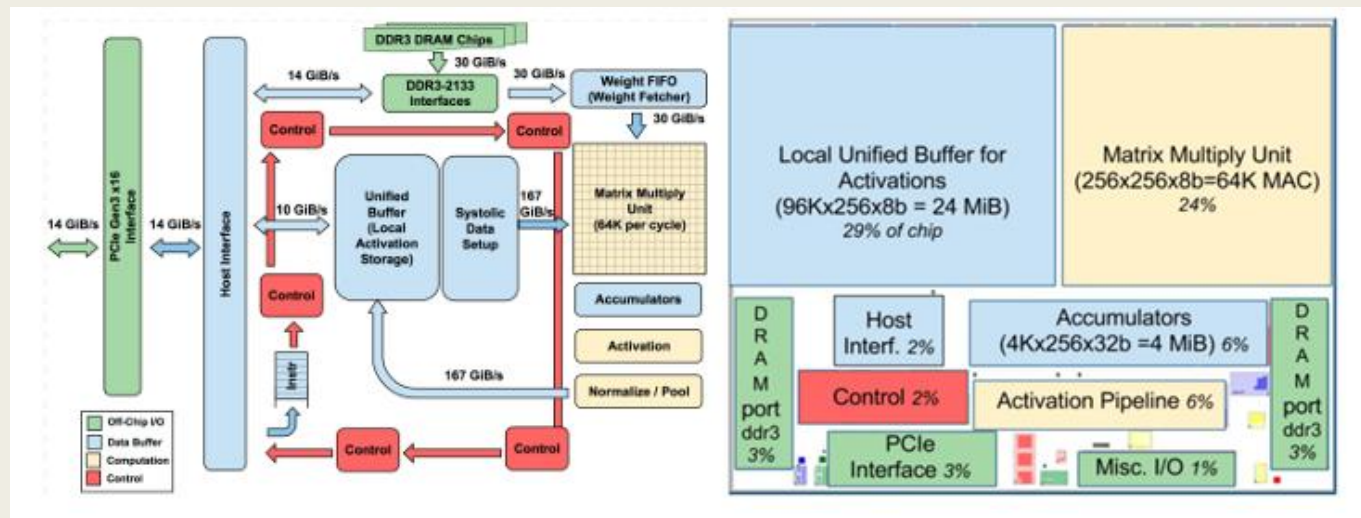
TPU 구조(Interface)

- Host Interface: TPU가 서버의 호스트 CPU와 소통하는 통로(Pcie 버스를 통해 명령어를 전달받고 연산할 데이터 주고받음)
- DMA Controller(Host Interface 내부): CPU 간섭 없이 호스트 메모리와 Unified Buffer 사이에서 데이터를 직접 전송하여 전송 속도 높임



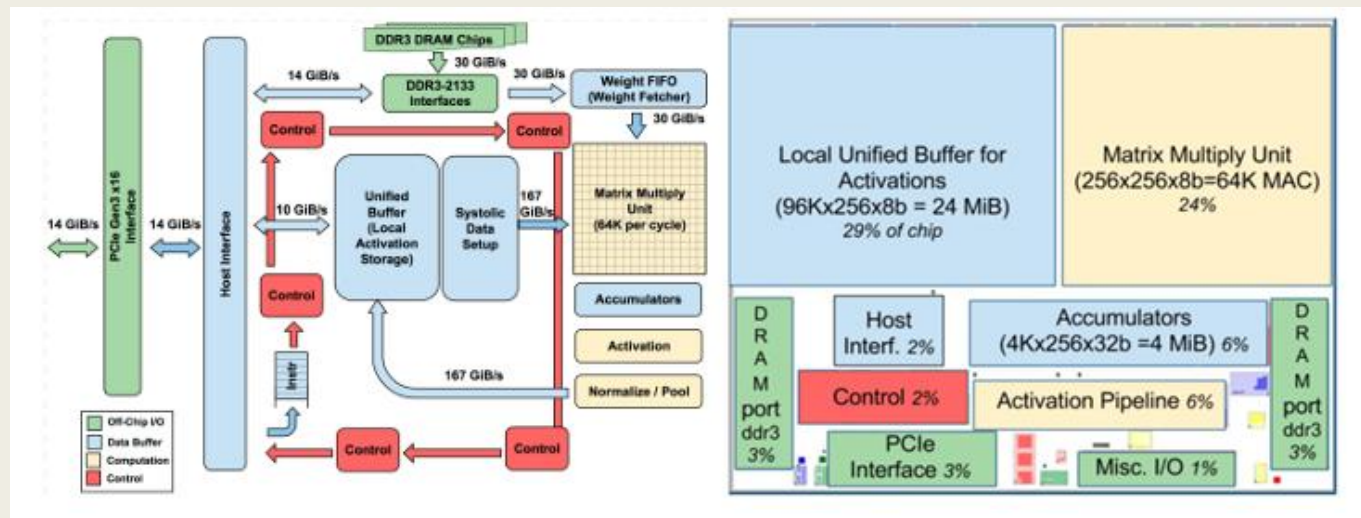
TPU 구조(Memory System)

- Unified Buffer: SRAM, 신경망의 입력 데이터 저장
- Weight Memory(DDR3): 칩 외부에 위치한 메모리, 신경망의 가중치 저장
- Weight FIFO: 외부 DDR3 메모리에서 읽어온 가중치를 MMU에 전달하기 전에 임시로 보관하는 버퍼



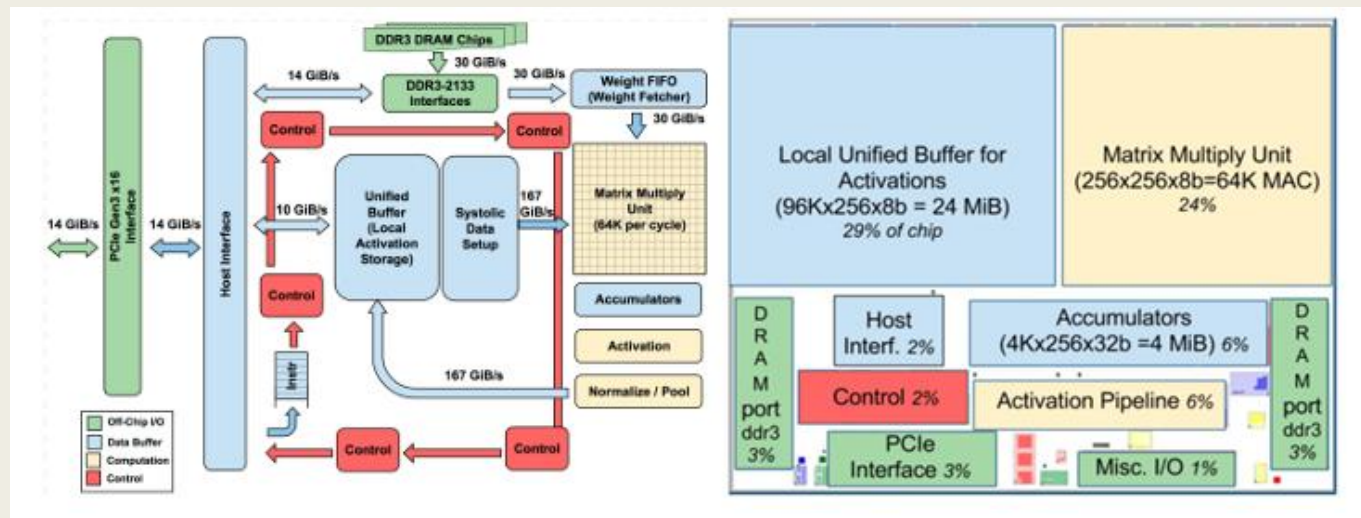
TPU 구조(Compute Unit)

- Matrix Multiply Unit(MMU):
TPU의 가장 중요한 부분으로, Buffer의 입력값과 FIFO의 가중치가 만나서 행렬 곱셈이 일어나는 곳
- Accumulator: MMU에서 계산된 16비트 결과값들을 더해서 32비트 정밀도로 저장하는 공간
- Activation Unit: Accumulator에 저장된 결과값에 ReLU, Sigmoid와 같은 비선형 함수를 적용하는 곳



TPU 구조(Control)

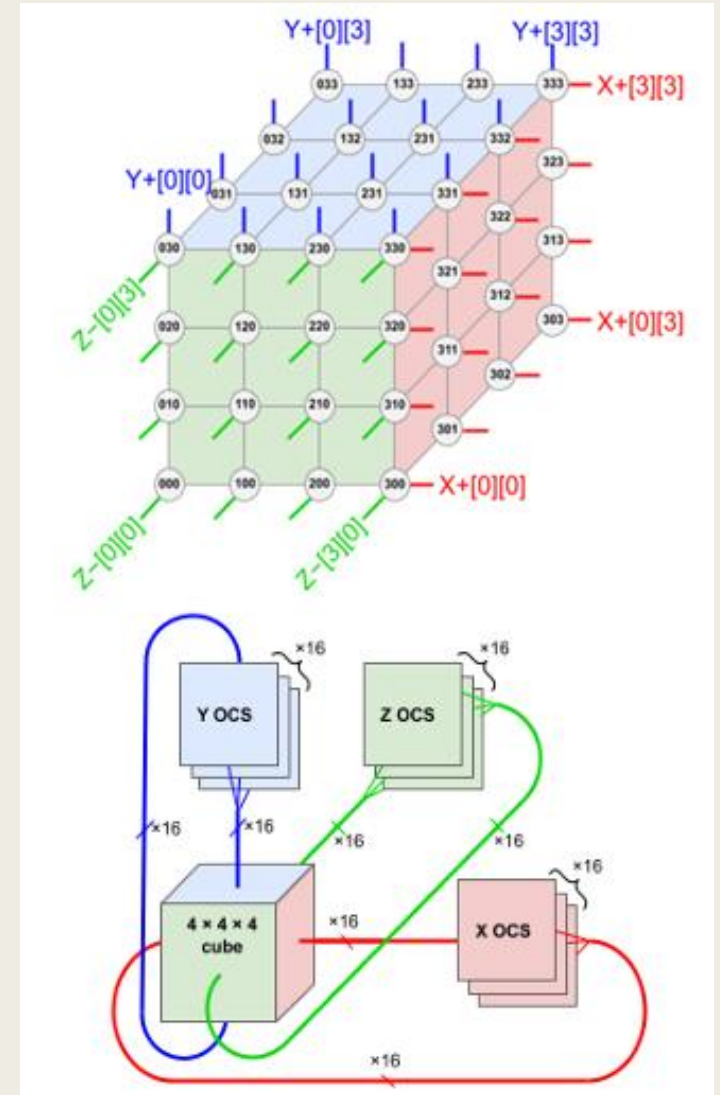
- Control Unit: 호스트 CPU로부터 받은 CISC 명령어를 해석하여 각 유닛에 동작 신호 보냄



Optical Circuit Switching

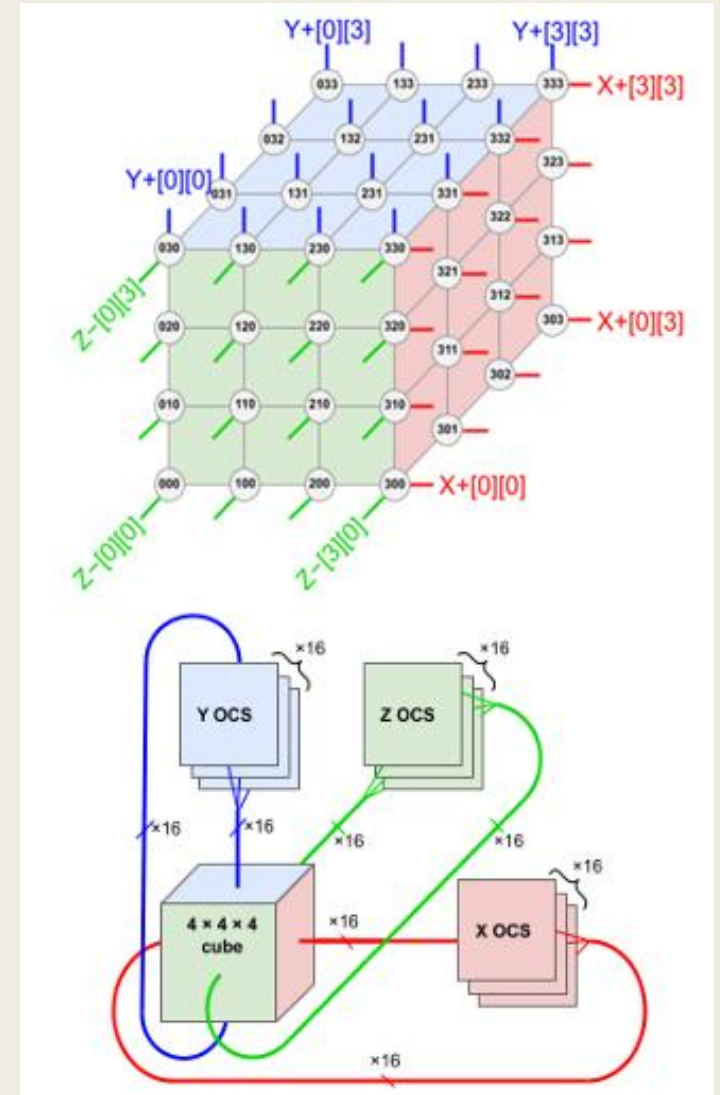
Optical Circuit Switching

- 도입 배경
기존 TPU v3는 2D 토러스 구조를 사용했는데, 시스템 규모를 4배로 키우면서 비용/가용성 문제가 생김
- 3D 토러스 구조를 채택하여 통로를 넓히고, OCS를 도입하여 고장 난 칩을 소프트웨어로 즉시 건너뛰게 할 수 있게 만듦
- 2D 토러스 vs 3D 토러스 (Bisection Bandwidth)
2D(가로, 세로)보다 3D(가로, 세로, 높이) 격자 구조로 연결하면 노드 간의 평균 거리가 짧아지고 이분 대역폭이 크게 늘어납니다. 즉, 데이터 병목 현상이 줄어들어 대규모 학습 효율이 극대화됩니다.



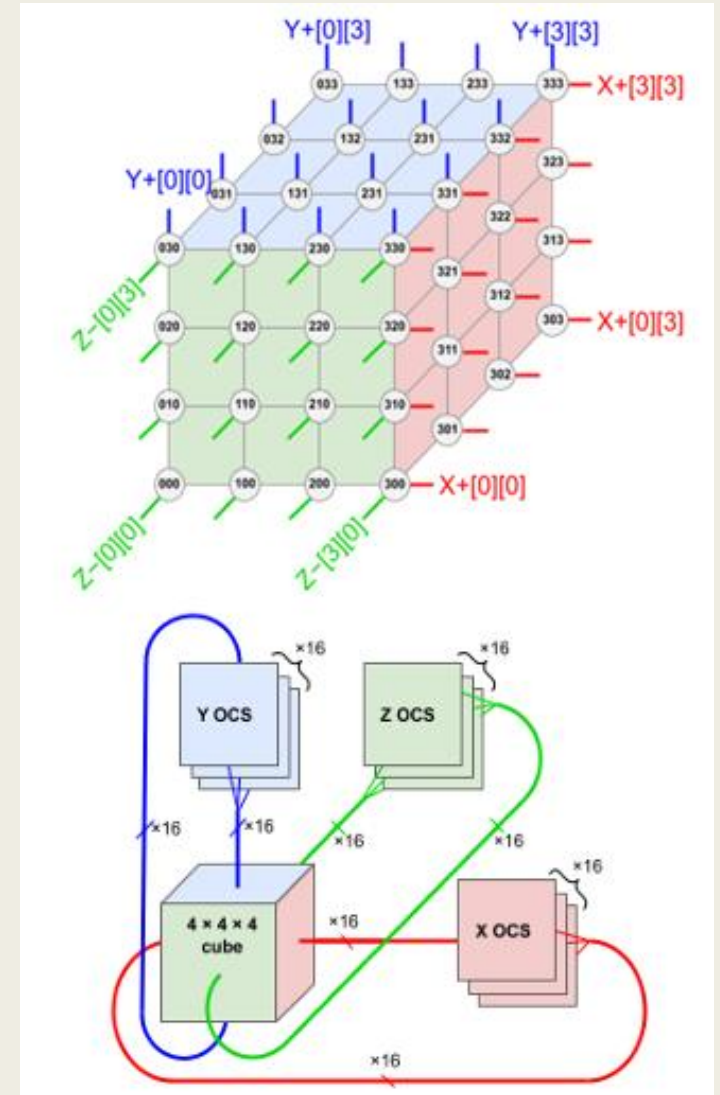
Optical Circuit Switching

- OCS의 도입
 1. OCS를 사용하면 비싼 광학 부품을 모든 곳에 도배할 필요 없이, 필요한 핵심 경로에만 효율적으로 배치할 수 있습니다. 이 광학 시스템은 전체 비용의 5% 미만으로 억제됨.
 2. 토폴로지 가변성: 사용자는 자신의 모델 특성에 맞춰 네트워크 모양을 최적의 상태(예: twisted 3D 토러스)로 설정할 수 있는 유연성까지 얻음



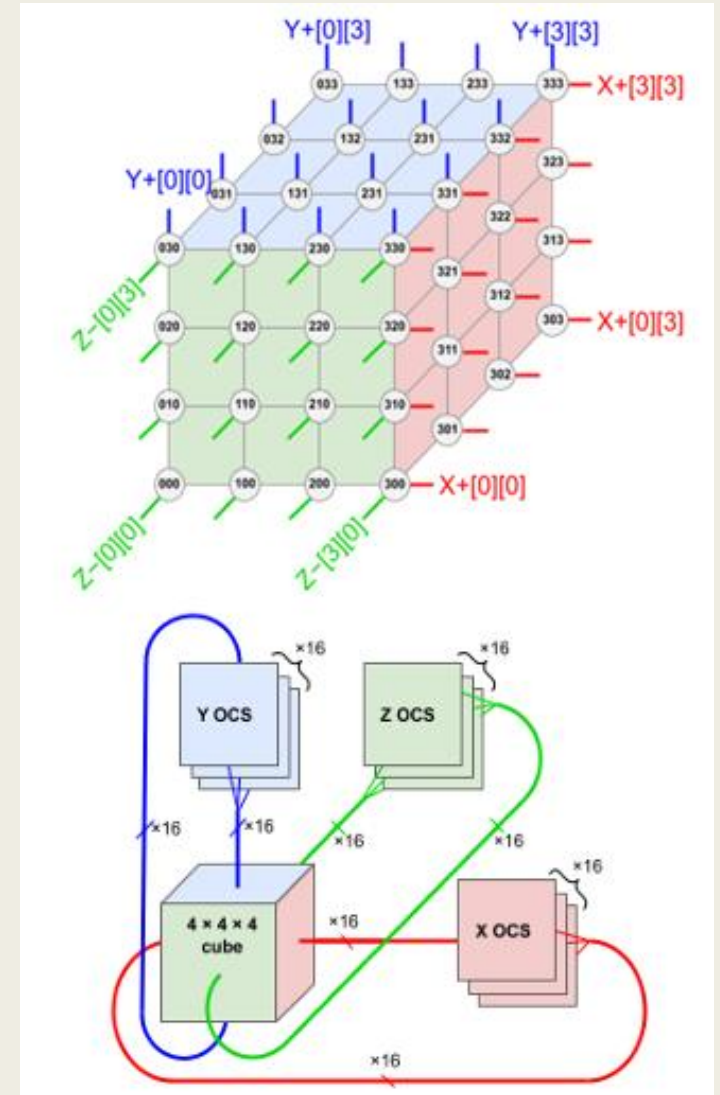
Optical Circuit Switching

- 상단 그림: 4x4x4 큐브
- 구성 요소: 64개의 TPU v4 칩과 이를 제어하는 16개의 CPU 호스트가 하나의 rack 안에 포함된 구조
- 6개의 면: 큐브의 6개 면에는 16개의 광학 링크가 각각 연결되어 총 96개의 광학 링크가 밖으로 나옴
- 좌표: 각 차원과 방향, 해당 면 내에서의 16개의 링크 위치를 나타내는 좌표



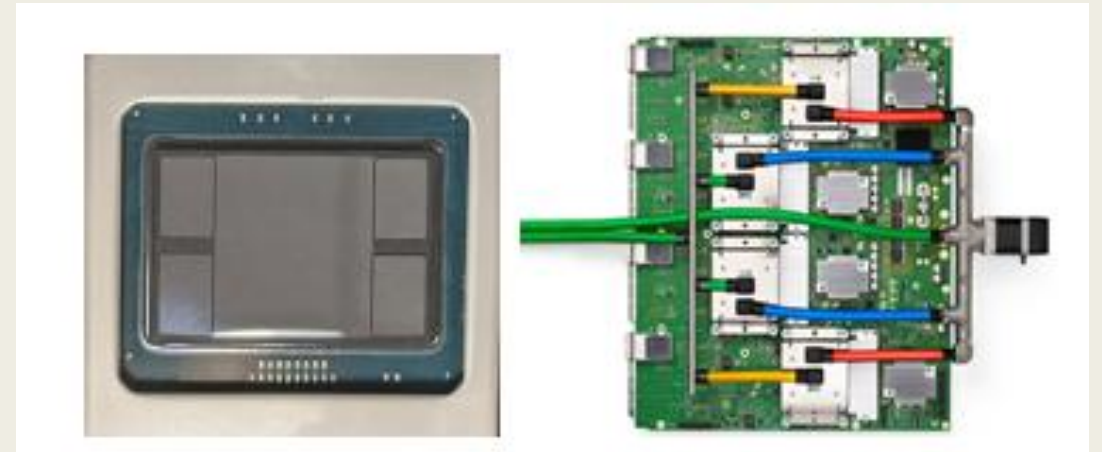
Optical Circuit Switching

- 하단 그림: 3종류의 OCS
- 큐브의 마주 보는 두면에서 나온 링크들이 동일한 OCS에 연결되어야 함
- 이 그림을 보면 하나의 4x4x4 블록은 총 48개의 OCS와 연결되며 이러한 연결을 통해 최종적으로 4096개의 칩이 하나의 거대한 슈퍼 컴퓨터 형성
(48이 나오는 과정: 총 외부 링크 수: $6 \times 16 = 96$, 마주 보는 면을 하나의 세트로 묶음 $\rightarrow 96/2 = 48$ 개의 케이블 쌍 $\rightarrow$ 서로 다른 48개의 OCS에 하나씩 연결)
- 좌표 하나하나가 개별 TPU 칩 하나하나에서 나오는 통신 회선



Optical Circuit Switching

- 좌측 그림: 개별 TPU v4 칩의 패키지
하나의 패키지 안에 2개의 TensorCore가 들음
칩 주변에는 4개의 HBM2 메모리가 배치됨
- 우측 그림: 4개의 TPU 패키지가 장착된 하나의
보드
4개의 칩이 하나의 기판 위에 2x2 형태로 연결됨 + 액체 냉각 판으로 덮여 열을 식힘



Optical Circuit Switching

- 8개의 큰 덩어리인 Rack으로 구성됨
이 8개의 랙 묶음에는 총 (8x64) 512개의 TPU 칩 포함됨
- 랙은 크게 세 부분으로 나뉨
상단: 16개의 호스트 서버
16대의 서버가 4개의 TPU 칩 관리
중간: 16개의 TPU v4 트레이(실질적 연산 엔진)
한 트레이 안에 4개의 TPU v4 칩 장착
총 칩 수: 16개 트레이 x 4개의 칩 = 64
배선/냉각: 내부 구리선/외부 광학선 + 파이프 라인 형태 액체 냉각 시스템



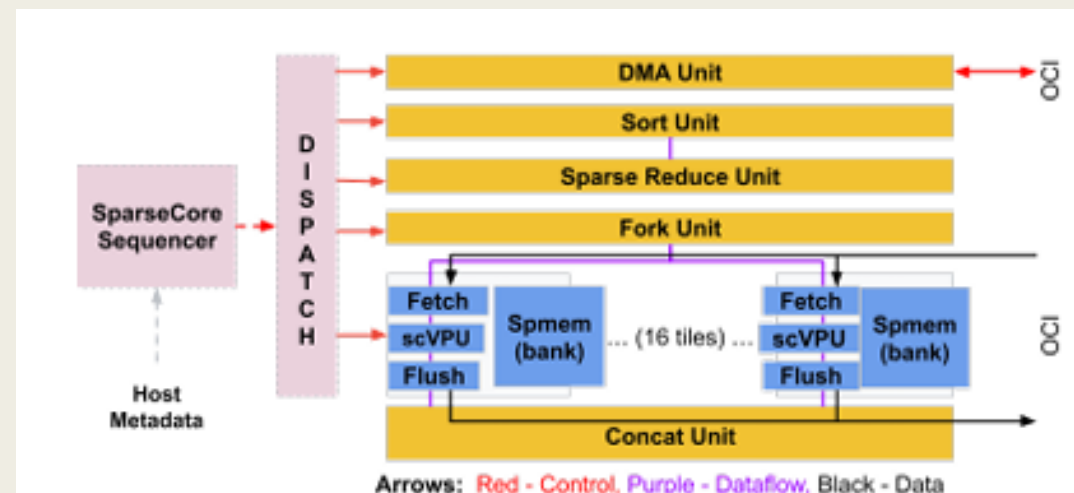
Optical Circuit Switching

- OCS 덕분에 칩들이 어디에 있든 광섬유로 묶어줄 수 있음
- 광학 스위치로 물리적 경로를 끊어버릴 수 있기 때문에, 보안성 또한 뛰어남
- 훈련 방식에 따라 길쭉한 모델/데이터가 많은 모델 과 같이 네트워크 모양을 바꿔 성능을 향상시킴
- 선을 살짝 꼬아서 데이터가 이동하는 지름길을 만드는 twisted 토러스 기술 이용
데이터 공유 속도가 빨라 짐
- 이 모든 것을 결합하여 아주 적은 투자로 엄청난 유연성과 성능 향상 얻음

SparseCore

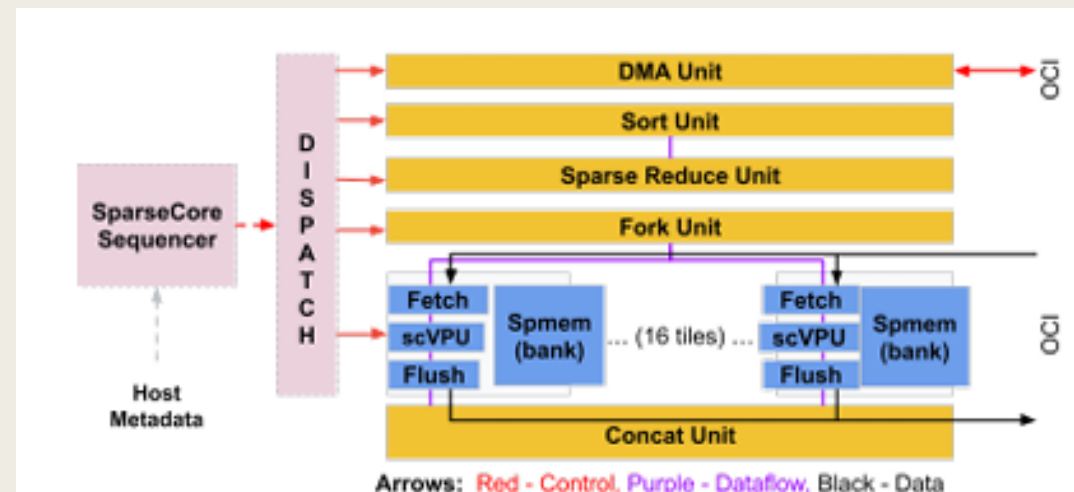
SparseCore

- DLRM은 구글 ML 워크로드의 1/4을 차지함, 임베딩은 DLRM의 핵심 구성 요소
- 임베딩: 범주형 특성(categorical features)으로 구성됨/범주형 특성 값을 조밀한 벡터로 변환하는 표준적이고 효과적인 방법
- 임베딩 함수는 룩업 테이블을 사용하여 구현
- 임베딩 테이블은 매우 커서 여러 TPU 칩의 메모리에 분할되어 저장
 1. column sharding
 2. row sharding
 3. table sharding
- 이러한 분산 전략 -> 모델 병렬성
- 임베딩 룩업 연산은 주로 작은 gather 또는 scatter 메모리 액세스로 구성됨.



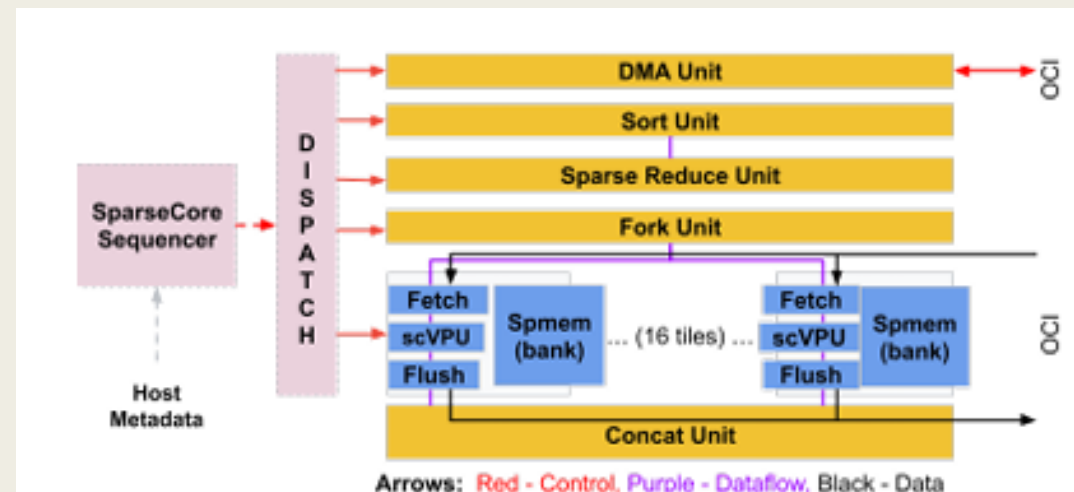
SparseCore

- Host Metadata: 어떤 임베딩 테이블에서 어떤 인덱스를 가져와야 하는지에 대한 정보 담김
- SparseCore Sequencer: Host Metadata 해석 -
> 각 연산 타일과 특수 유닛이 언제 움직여야 할지 제어 신호 보냄
- Dispatch DMA: HBM, ICI에서 데이터를 가져오거나 보낼 때, 정확한 위치로 파견하는 역할



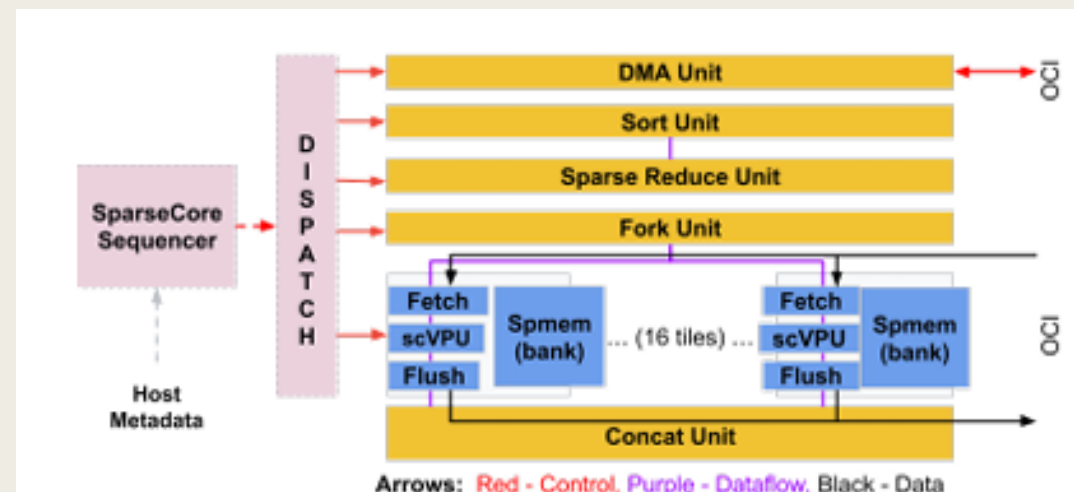
SparseCore

- Fetch Unit: 메모리에서 임베딩 벡터나 중간 데이터를 읽어옴
- scVPU: SC의 핵심 계산기, 8-와이드 SIMD(한 번에 8개의 숫자를 계산) 구조로, 임베딩 연산에 필요한 가벼운 벡터 계산을 수행
- Spmem: 각 타일이 사용하는 고속 전용 메모리(SRAM)
- Flush Unit: 계산이 끝난 결과를 다시 외부 메모리나 다음 단계로 밀어냄



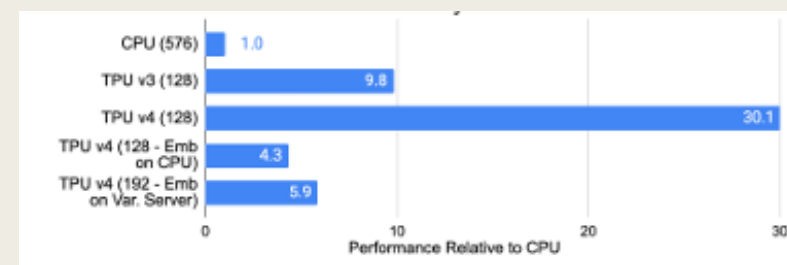
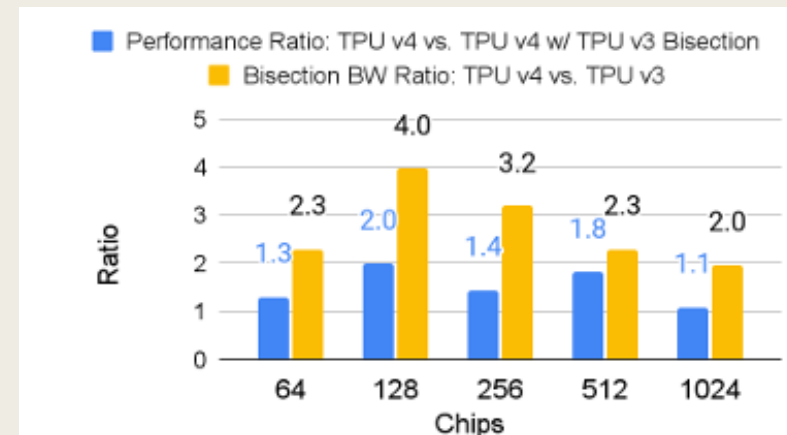
SparseCore

- Sort Unit: 임베딩 인덱스를 정렬
- Sparse Reduce Unit: 여러 개의 임베딩 벡터를 하나로 합치는(Sum, Mean 등) 리덕션 연산을 전
- Fork Unit: 데이터를 여러 갈래로 나눔
- Concat Unit: 흩어진 결과물을 하나로 이어 붙임



SparseCore

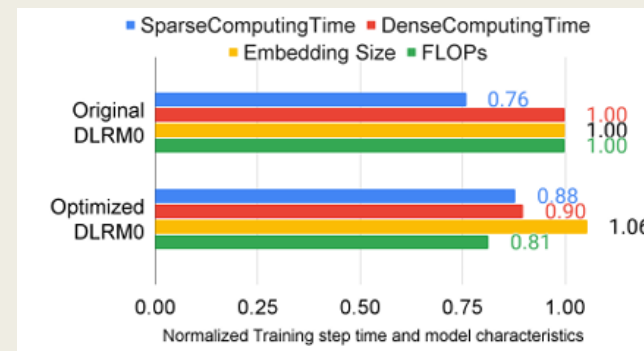
- 독립된 CPU 구성보다 성능이 좋고 빠름을 보여주고, 이전 모델들보다도 성능이 좋음을 확인할 수 있음



TPU의 성능 관련 지표

ML을 사용하여 DNN을 TPU에, TPU 토폴로지를 DNN에 최적화하기

- PA-NAS (Platform-Aware Neural Architecture Search): 이 모델이 TPU v4라는 특정 기계에서 얼마나 빨리 돌아가는가?"까지 고려하여 모델의 구조를 짜는 기술
- DLRM(추천 모델)은 임베딩 전용 코어(SC)와 계산 전용 코어(TC)를 같이 씁니다. 그런데 한쪽이 일을 다 끝냈는데 다른 쪽이 아직 일하고 있으면 노는 코어가 생깁니다(병목 현상) -> PA-NAS가 모델의 층(layer) 구성을 바꿔서 SC와 TC가 거의 동시에 일을 끝내도록 맞춤
- 맞춤형 모양: 정육면체 혹은 길쭉한 직사각형으로 묶을지에 따라 성능 천차만별 -> 초보자가 설정한 것보다 2.3배 빨라짐
- TPU v4 시스템의 진정한 위력은 하드웨어 자체 뿐만 아니라, 그 하드웨어의 복잡한 연결 구조와 모델의 내부 구조를 AI(PA-NAS)를 통해 자동 최적화하여 성능을 극한까지 끌어올리는 소프트웨어 지능



Case	Versions	Through-put (seqs/sec)	Hyper-Parameters (topology, partition spec [pipeline, data, model1, model2], 1D/2D activation/weight partitioning)
LLM	Novice's pick	17.9 (1.0x)	4×8×16, [1, 1, 16, 32], 2D/2D
	Best perf.	41.3 (2.3x)	8×8×8, [1, 1, 64, 8], 1D/2D
GPT-3 Pre-training	Expert's pick	21.0 (1.0x)	8×8×8, [8, 1, 8, 8], 2D/2D
	Best perf.	25.0 (1.2x)	4×8×16, [16, 4, 1, 8], 1D/1D

Production Workload Performance

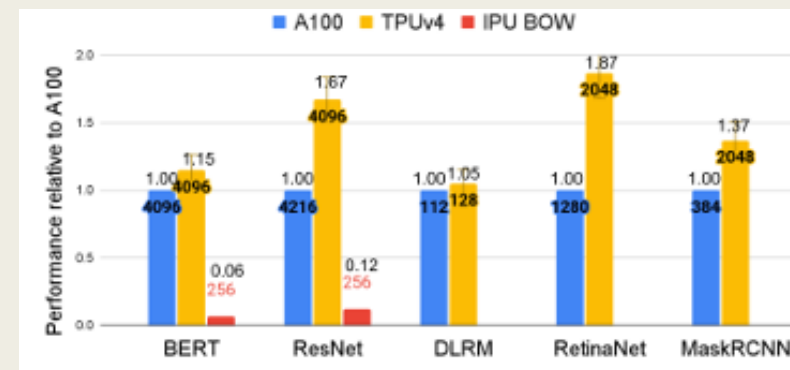
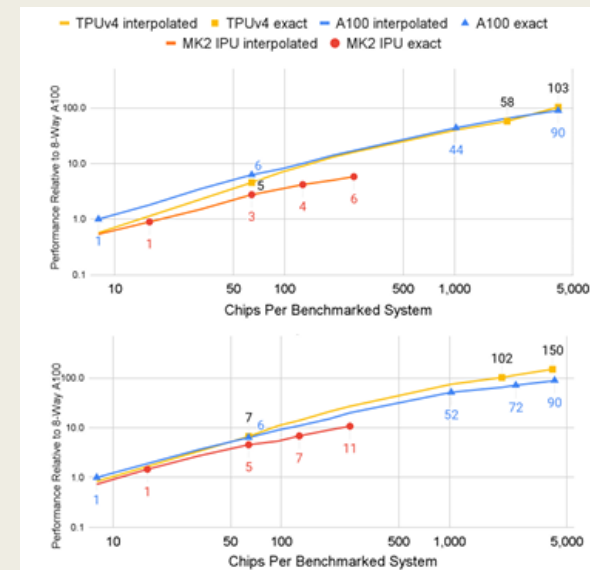
- TPU v4는 실제 구글 서비스 환경에서 압도적인 효율을 보여준다
- 제조 공정에서 v3는 16nm인 반면 v4는 7nm로 공정 밀도 향상 덕에 행렬 곱셈기가 두 배로 늘어남
- 하드웨어 부분에서 V3에서 v4로 넘어가면서 16nm에서 7nm로 넘어가며 칩 안에 더 많은 행렬 곱셈기를 넣을 수 있게 되었고, 성능이 2.2배 뛰어남. 또한 와트당 성능도 2.7배 좋아짐
- 기존 HBM에 비해 훨씬 가깝고 빠른 비상용 메모리 공간인 CMEM이 나왔다
- SparseCore 덕분에 훈련 속도가 v3 대비 3배 이상 빨라짐
- v3는 2D로 네트워크가 고정되어 있어 LLM을 쪼개서 훈련하기 비효율적이지만 v4는 3D/OCS 유연한 네트워크가 있어야 최신 LLM을 제대로 훈련시킬 수 있다는 점을 강조
- 경쟁사인 Nvidia A100도 매우 강력한 칩이지만, 구글 TPU v4가 전력은 훨씬 적게 쓰면서도 성능은 동등하거나 그 이상을 주장 -> 데이터 센터 비용 측면에서 차이 만둠

	Google TPUv4	TPUv3
Production deployment	2020	2018
Peak TFLOPS	275 (bf16 or int8)	123 (bf16)
Clock Rate	1050 MHz	940 MHz
Tech. node, Die size	7 nm, <600 mm ²	16 nm, < 700 mm ²
Transistor count	22 billion	10 billion
Chips per CPU host	4	8
TDP	N.A.	N.A.
Idle, min/mean/max power	90, 121/170/192 W	123, 175/220/262 W
Inter Chip Interconnect	6 links @ 50 GB/s	4 links @ 70 GB/s
Largest scale configuration	4096 chips	1024 chips
Processor Style	Single Instruction 2D Data	Single Instruction 2D Data
Processors / Chip	2	2
Threads / Core	1	1
SparseCores / Chip	4	2
On Chip Memory	128 (CMEM) + 32 MiB (VMEM) + 10 MiB (spMEM)	32 MiB (VMEM) + 5 MiB (spMEM)
Register File Size	0.25 MiB	0.25 MiB
HBM2 capacity, BW	32 GiB, 1200 GB/s	32 GiB, 900 GB/s

	Nvidia A100	Graphcore MK2 IPU
Production deployment	2020	2021
Peak TFLOPS	312 (bf16), 624 (i8)	250 (bf16)
Clock Rate Base/Boost	1095 /1410 MHz	1850 MHz
Tech. node, Die size	7 nm, 826 mm ²	7 nm, 832 mm ²
Transistor count	54 billion	59 billion
Chips per CPU host	4	4
TDP	400 W	300 W
Inter Chip Interconnect	12 links @ 25 GB/s	3 links @ 64 GB/s
Largest scale MLPerf 2.0 configuration	4216 chips	256 chips
Processor Style	Single Instruction Multiple Threads	Multiple Instruction Multiple Data
Processors / Chip	108	1472
Threads / Core	32	6
On Chip Memory	40 MiB	900 MiB
Register File Size	27 MiB	1.40 MiB
HBM2 capacity, BW	80 GiB, 2039 GB/s	0

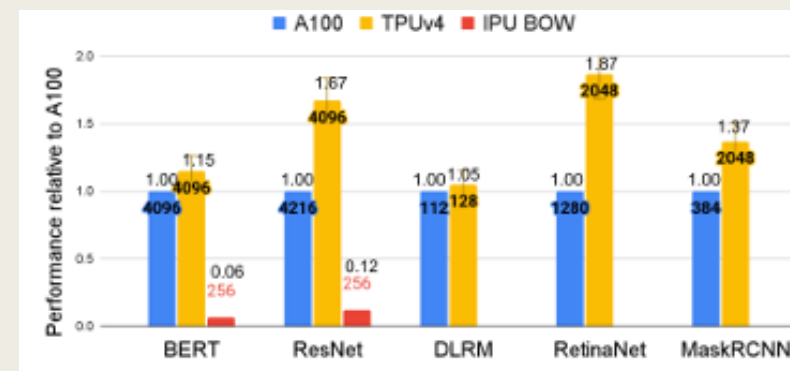
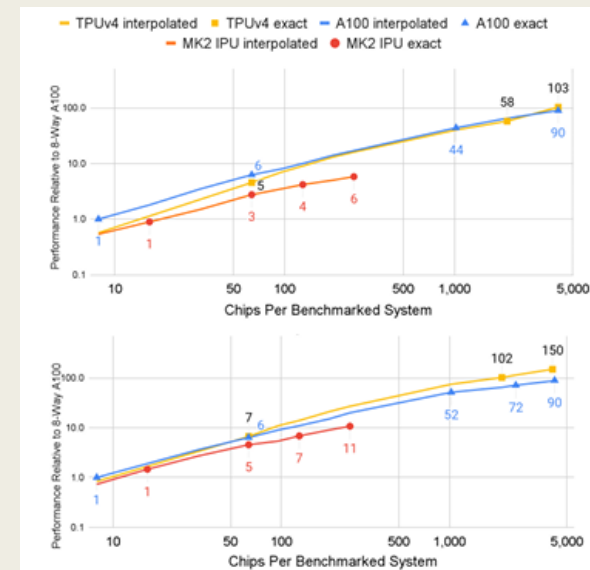
MLPerf Benchmark Performance

- Nvidia A100과 Graphcore MK2 IPU라는 두 가지 후보의 특징을 비교
- Google TPU v4: 스레드 수는 적지만, 행렬 연산(MXU)과 임베딩(SC)에 특화된 구조입니다. 칩 크기도 경쟁사보다 40% 작아 경제적
- Nvidia A100: 수천 개의 스레드와 거대한 레지스터 파일을 통해 어떤 작업이든 유연하게 처리할 수 있도록 설계
- Graphcore IPU: 외부 메모리(HBM)를 아예 버리고 칩 안에 모든 데이터를 넣으려 시도했습니다. 하지만 이 때문에 시스템 확장성(Scalability)에서 한계



MLPerf Benchmark Performance

- 성능(BERT& Resnet)
- BERT: TPU v4는 A100과 비슷한 규모에서 약 1.15배 더 빠름
- ResNet (이미지 모델): TPU v4가 A100보다 1.67 배 더 빠름
- 구글과 엔비디아는 4,000개 이상의 칩을 묶어 거대 시스템을 구축할 수 있음을 증명했지만, 그래프코어는 256개 칩 수준에 머물렀음
- 동일한 작업을 수행할 때, Nvidia A100은 TPU v4보다 전력을 최대 1.9배(거의 2배) 더 많이 씁니다.



Summary

Summary

- TPU v4의 두 가지 주요 아키텍처 기능은 적은 비용으로 엄청난 이점을 제공
- SparseCore는 임베딩을 TPU v4 슈퍼컴퓨터의 128 TiB 물리 메모리 어디에나 배치할 수 있게 해주는 데이터플로우 '코어의 바다(sea-of-cores)' 아키텍처를 제공함으로써 DLRM 모델의 임베딩을 5~7배 가속화합니다. 이 이득은 다이 면적과 전력의 약 5%라는 비용만으로 얻어진다.
- OCS 또한 확장성, 개선된 가용성, 모듈성, 높은 성능, 전력 감소, 스케줄링 단순화, 빠른 배포, 보안 강화와 같은 8가지 이점을 제공한다.
- 이 기능들을 적용한 TPU v4는 훈련 시간은 TPU v3 대비 크게 단축되고, 성능과 확장성 및 가용성은 높였으며, 에너지 소비량도 감소하여 놀라운 잠재력을 가졌음을 보인다. -> 지속 가능한 AI

TPU vs DFX

	TPU v4	DFX
하드웨어 유형	ASIC 기반 DSA	멀티 FPGA 기반 어플라이언스
핵심 가속기	SparseCore	Custom Decoder Core
상호 연결성	광학 회로 스위치 (OCS)	링 네트워크
확장 규모	4096개의 칩을 연결한 슈퍼컴퓨터	4개의 FPGA 탑재한 서버 어플라이언스